

HILL CIPHER (ENCRPTION AND DECRPTION)

PROJECT PROPOSAL

SUBMITTED TO:

SIR ADEEL AMJAD

BS ARTIFICIAL INTELLIGENCE



SUPERIOR UNIVERSITY

DEPARTMENT OF SOFTWARE ENGINEERING

FACULTY OF COMPUTER SCIENCE

SUBMITTED BY:

MINAHIL IQBAL

062

BSAI-6B

PROJECT PROPOSAL

Hill Cipher Encryption & Decryption using 8086 Assembly Language

1. Project Title

Implementation of Hill Cipher Algorithm using 8086 Assembly Language

2. Introduction

The Hill Cipher is a classical encryption technique based on linear algebra concepts. It uses matrix multiplication to encrypt plaintext into ciphertext, making it more secure than simple substitution ciphers.

This project aims to implement the Hill Cipher algorithm using 8086 Assembly Language, providing low-level insight into encryption mechanisms and processor operations.

3. Objectives

- To understand the working of classical cryptographic techniques
- To implement matrix-based encryption in 8086 Assembly
- To perform both encryption and decryption of text
- To strengthen concepts of registers, memory, loops, and arithmetic operations in assembly

4. Scope of the Project

- Encrypt plaintext using Hill Cipher (2×2 matrix)
- Decrypt ciphertext back to original text
- Handle uppercase alphabets (A–Z)
- Perform modular arithmetic (mod 26)
- Work within 8086 register limitations

5. Tools & Technologies

- **Language:** 8086 Assembly Language
- **Assembler:** NASM / MASM / TASM
- **Platform:** DOS-based environment

Concepts Used:

- Registers (AX, BX, CX, DX)
- Memory addressing
- Loops and conditional jumps
- Arithmetic operations

6. Methodology

◆ Step 1: Input

Take plaintext from user

Convert characters (A=0, B=1, ... Z=25)

◆ Step 2: Key Matrix

Define a 2×2 key matrix

Ensure determinant is invertible (for decryption)

◆ Step 3: Encryption

Divide plaintext into pairs

Apply matrix multiplication

Take modulo 26

Convert numbers back to characters

◆ Step 4: Decryption

Find inverse of key matrix

Apply same multiplication process

Recover original text

7. Expected Output

Encrypted message (ciphertext)

Decrypted message (original plaintext)

Correct transformation using Hill Cipher logic

8. Challenges

- Implementing matrix multiplication in assembly
- Handling modular arithmetic (mod 26)
- Finding inverse matrix in assembly
- Managing limited registers

9. Applications

- Basic data security concepts
- Educational cryptography systems
- Understanding modern encryption foundations

10. Conclusion

This project will demonstrate how encryption algorithms can be implemented at a low level using 8086 Assembly Language. It will enhance understanding of both assembly programming and cryptographic techniques, especially matrix-based encryption like Hill Cipher.

11. Future Enhancements

- Support larger matrices (3×3)
- Add lowercase and special characters
- Improve user interface
- Optimize performance

PROJECT CODE

```
;-----HILL CIPHER-----;
```

```
org 100h
```

```
start:
```

```
    call clrscr
```

```
    call print_banner
```

```
    mov si, msg_setup_key
```

```
    call print_str
```

```
    call do_set_key_internal
```

```
main_menu:
```

```
    call print_menu
```

```
    mov ah, 08h
```

```
    int 21h
```

```
    push ax
```

```
    mov dl, al
```

```
    mov ah, 02h
```

```
    int 21h
```

```
    call print_crlf
```

```
    pop ax

    cmp al, '1'

    je near do_encrypt

    cmp al, '2'

    je near do_decrypt

    cmp al, '3'

    je near quit

    jmp main_menu

quit:

    mov ah, 4Ch

    int 21h

; ----- KEY SETUP -----

do_set_key_internal:

    mov si, msg_k1

    call print_str

    call read_num

    mov [k1], al

    mov si, msg_k2

    call print_str

    call read_num

    mov [k2], al

    mov si, msg_k3
```

```
    call print_str
    call read_num
    mov [k3], al
    mov si, msg_k4
    call print_str
    call read_num
    mov [k4], al
    call calc_inverse
    ret

; ----- ENCRYPT -----

do_encrypt:
    mov si, msg_enter_plain
    call print_str
    call read_input
    call clean_input
    mov di, out_buf
    mov si, work_buf
    mov cx, [work_len]

enc_loop:
    cmp cx, 0
    jbe enc_fin
    mov bl, [si]
```

```
    mov bh, 'X'-'A'

    cmp cx, 1

    je .calc

    mov bh, [si+1]

.calc:

    push bx

    mov al, [k1]

    mul bl

    mov dx, ax

    mov al, [k2]

    mul bh

    add ax, dx

    mov dl, 26

    div dl

    mov [di], ah

    inc di


    pop bx

    mov al, [k3]

    mul bl

    mov dx, ax

    mov al, [k4]
```



```
mul bh
```

```
add ax, dx
```

```
mov dl, 26
```

```
div dl
```

```
mov [di], ah
```

```
inc di
```

```
add si, 2
```

```
sub cx, 2
```

```
jmp enc_loop
```

```
enc_fin:
```

```
mov ax, di
```

```
sub ax, out_buf
```

```
mov [out_len], ax
```

```
call show_and_save
```

```
jmp main_menu
```

```
; ----- DECRYPT -----
```

```
do_decrypt:
```

```
mov si, msg_enter_cipher
```

```
call print_str
```

```
call read_input
```

```
call clean_input
```

```
    mov di, out_buf

    mov si, work_buf

    mov cx, [work_len]

dec_loop:

    cmp cx, 0

    jbe dec_fin

    mov bl, [si]

    mov bh, 0

    cmp cx, 1

    je .calc

    mov bh, [si+1]

.calc:

    push bx

    mov al, [ik1]

    mul bl

    mov dx, ax

    mov al, [ik2]

    mul bh

    add ax, dx

    mov dl, 26

    div dl

    mov [di], ah
```

inc di

pop bx

mov al, [ik3]

mul bl

mov dx, ax

mov al, [ik4]

mul bh

add ax, dx

mov dl, 26

div dl

mov [di], ah

inc di

add si, 2

sub cx, 2

jmp dec_loop

dec_fin:

mov ax, di

sub ax, out_buf

mov [out_len], ax

call show_and_save

```
    jmp main_menu

; ----- HELPERS -----

show_and_save:

    mov si, msg_result

    call print_str

    mov si, out_buf

    mov cx, [out_len]

.p:

    lodsb

    add al, 'A'

    mov dl, al

    push ax

    mov ah, 02h

    int 21h

    pop ax

    loop .p

    call print_crlf


    mov si, msg_save

    call print_str

    mov ah, 01h

    int 21h
```

```
    push ax
    call print_crlf
    pop ax
    cmp al, 'y'
    je .do_save
    cmp al, 'Y'
    jne .skip
.do_save:
    mov dx, filename
    mov cx, 0
    mov ah, 3Ch
    int 21h
    jc .skip
    mov bx, ax
    mov si, out_buf
    mov di, work_buf
    mov cx, [out_len]
.conv:
    lodsb
    add al, 'A'
    stosb
    loop .conv
```

```
    mov dx, work_buf  
    mov cx, [out_len]  
    mov ah, 40h  
    int 21h  
    mov ah, 3Eh  
    int 21h  
.skip:  
    ret
```

```
calc_inverse:
```

```
    mov al, [k1]  
    mul byte [k4]  
    mov bx, ax  
    mov al, [k2]  
    mul byte [k3]  
    sub bx, ax
```

```
.adj:
```

```
    cmp bx, 0  
    jge .mod  
    add bx, 26  
    jmp .adj
```

```
.mod:
```

```
    mov ax, bx
    mov dl, 26
    div dl
    mov bl, ah
    mov cl, 1
.inv_l:
    mov al, bl
    mul cl
    mov dl, 26
    div dl
    cmp ah, 1
    je .found
    inc cl
    cmp cl, 26
    jne .inv_l
    ret
.found:
    mov al, [k4]
    mul cl
    mov dl, 26
    div dl
    mov [ik1], ah
```

```
mov al, 26  
sub al, [k2]  
mul cl  
mov dl, 26  
div dl  
mov [ik2], ah  
mov al, 26  
sub al, [k3]  
mul cl  
mov dl, 26  
div dl  
mov [ik3], ah  
mov al, [k1]  
mul cl  
mov dl, 26  
div dl  
mov [ik4], ah  
ret
```

read_num:

```
mov ah, 01h  
  
int 21h
```



```
sub al, '0'
```

```
push ax
```

```
call print_crlf
```

```
pop ax
```

```
ret
```

```
clean_input:
```

```
mov cx, [input_len]
```

```
mov si, input_buf+2
```

```
mov di, work_buf
```

```
xor bx, bx
```

```
.l:
```

```
jcxz .d
```

```
lodsb
```

```
dec cx
```

```
and al, 0DFh
```

```
cmp al, 'A'
```

```
jb .l
```

```
cmp al, 'Z'
```

```
ja .l
```

```
sub al, 'A'
```

```
stosb
```

```
inc bx
```

```
jmp .l
```

```
.d:
```

```
mov [work_len], bx
```

```
ret
```

```
print_str:
```

```
mov dx, si
```

```
mov ah, 09h
```

```
int 21h
```

```
ret
```

```
print_crlf:
```

```
mov ah, 02h
```

```
mov dl, 13
```

```
int 21h
```

```
mov dl, 10
```

```
int 21h
```

```
ret
```

```
read_input:
```

```
mov dx, input_buf
```

```
mov byte [input_buf], 80
```

```
mov ah, 0Ah
```

```
int 21h
```

```
call print_crlf
```

```
mov bl, [input_buf+1]
```

```
mov bh, 0
```

```
mov [input_len], bx
```

```
ret
```

```
clrscr:
```

```
mov ax, 0600h
```

```
mov bh, 07h
```

```
mov cx, 0000h
```

```
mov dx, 184Fh
```

```
int 10h
```

```
ret
```

```
print_banner:
```

```
mov si, msg_banner
```

```
call print_str
```

```
ret
```

```
print_menu:
```

```
mov si, msg_menu
```

```
call print_str
```

```
ret
```

; ----- DATA -----

msg_banner db "HILL CIPHER SYSTEM", 13, 10, "\$"

msg_setup_key db "ENTER 2x2 KEY MATRIX:", 13, 10, "\$"

msg_menu db 13, 10, " [1] Encrypt", 13, 10, " [2] Decrypt", 13, 10, " [3] Quit", 13, 10, "Choice: \$"

msg_k1 db "k1: \$"

msg_k2 db "k2: \$"

msg_k3 db "k3: \$"

msg_k4 db "k4: \$"

msg_save db "Save result to result.txt? (y/n): \$"

filename db "result.txt", 0

msg_enter_plain db 13, 10, "Plaintext: \$"

msg_enter_cipher db 13, 10, "Ciphertext: \$"

msg_result db "Result: \$"

k1 db 0

k2 db 0

k3 db 0

k4 db 0

ik1 db 0

ik2 db 0

ik3 db 0

ik4 db 0

```
input_len dw 0

work_len dw 0

out_len dw 0

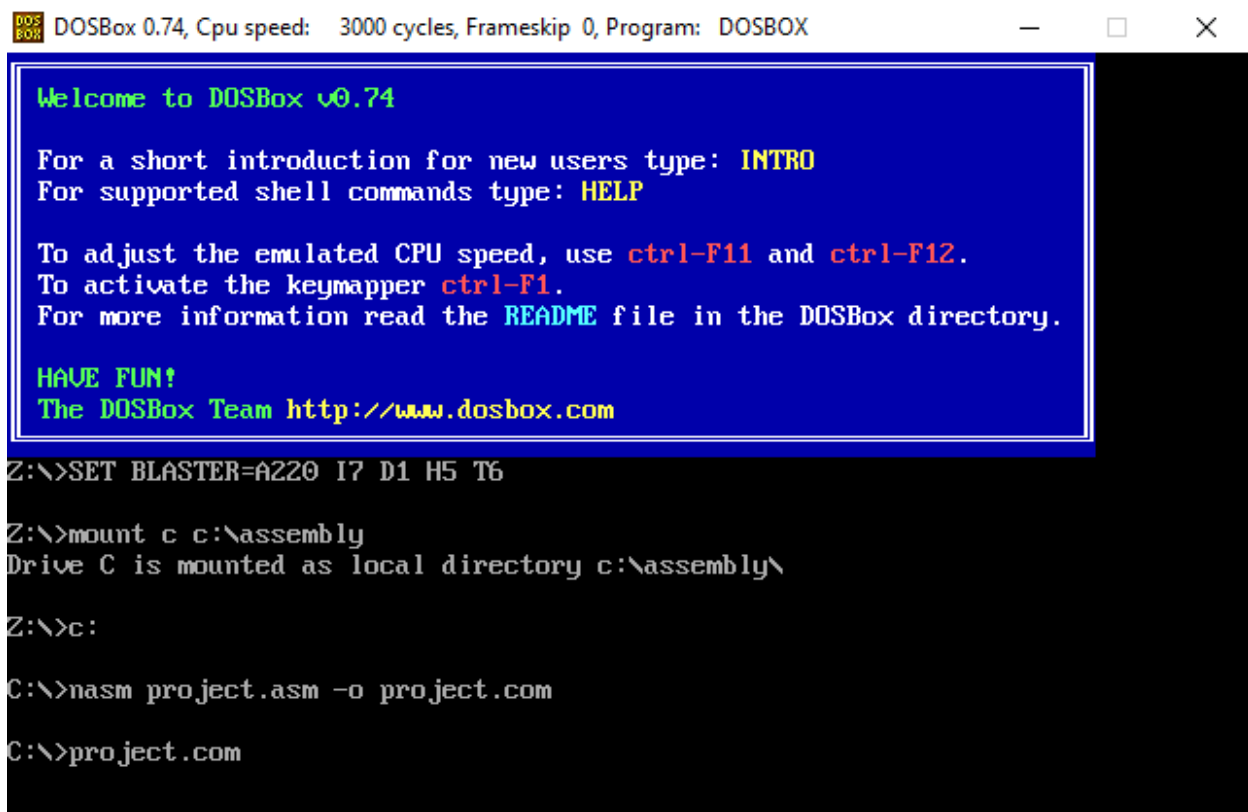
input_buf db 80, 0

        times 82 db 0

work_buf times 80 db 0

out_buf times 80 db 0
```

PROJECT OUTPUT



The screenshot shows a DOSBox 0.74 window. The title bar reads "DOSBox 0.74, Cpu speed: 3000 cycles, Frameskip 0, Program: DOSBOX". The main window has a blue background with white text. It displays a welcome message and instructions for new users. Below this, a command prompt session is shown with the following commands and output:

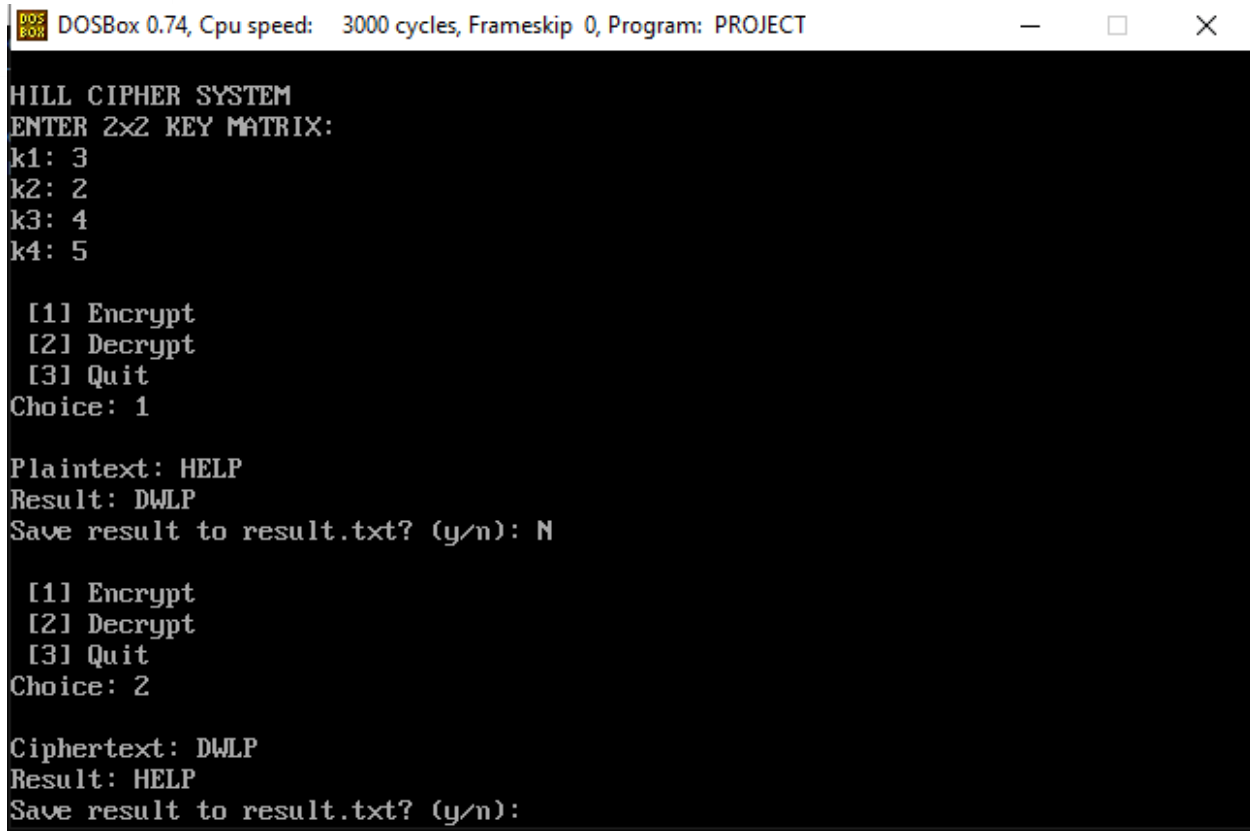
```
Z:\>SET BLASTER=A220 I7 D1 H5 T6

Z:\>mount c c:\assembly
Drive C is mounted as local directory c:\assembly\

Z:\>c:

C:\>nasm project.asm -o project.com

C:\>project.com
```



The screenshot shows a DOSBox window titled "DOSBox 0.74, Cpu speed: 3000 cycles, Frameskip 0, Program: PROJECT". The window contains a text-based program for a Hill Cipher system. The program prompts the user to enter a 2x2 key matrix with values k1, k2, k3, and k4. It then presents a menu with options: [1] Encrypt, [2] Decrypt, and [3] Quit. The user selects option 1 (Encrypt) and enters the plaintext "HELP". The program outputs the result "DWLP" and asks if the user wants to save the result to "result.txt". The user responds with "N". The program then presents the same menu again. The user selects option 2 (Decrypt). The program outputs the ciphertext "DWLP", the result "HELP", and asks if the user wants to save the result to "result.txt".

```
DOSBox 0.74, Cpu speed: 3000 cycles, Frameskip 0, Program: PROJECT

HILL CIPHER SYSTEM
ENTER 2x2 KEY MATRIX:
k1: 3
k2: 2
k3: 4
k4: 5

[1] Encrypt
[2] Decrypt
[3] Quit
Choice: 1

Plaintext: HELP
Result: DWLP
Save result to result.txt? (y/n): N

[1] Encrypt
[2] Decrypt
[3] Quit
Choice: 2

Ciphertext: DWLP
Result: HELP
Save result to result.txt? (y/n):
```